

Machine Learning Analysis of Nifty 50

Using Return-Based Modelling

A Structured Study on Model Behaviour, Stability, and Reliability

Siddhartha Jain

Siddharthajain956@gmail.com

2025-2026

Contents

1. Project Overview	1
2. Problem Statement	1
3. Dataset and Project Setup.....	2
4. Analytical Path / Project Method	3
5. Major Question 1 — Regression Baseline	5
6. Major Question 2 — Logistic Classification	7
7. Major Question 3 — Shallow Boosted Tree (Depth = 3).....	9
8. Major Question 4 — Ensemble Complexity	12
9. Major Question 5 — Evaluation & Reliability	14
10. Final Model Selection.....	16
11. Final Metrics Snapshot.....	17
12. Overall Interpretation	18
13. Limitations	19
14. Future Improvements.....	19
15. Final Answer to the Problem Statement.....	20
16. Appendix / Note on Supporting Assets.....	21

1. Project Overview

This project started with one basic question in my mind: can daily financial market data actually be understood properly through machine learning, or does it only start looking useful when we see some good-looking output on the screen? I did not want this project to become one more model comparison exercise where a few algorithms are applied and then judged only through scores. For me, that was never the real point. The real point was to see whether ML was actually reading something meaningful from the data, and whether that meaning could still remain when the model was tested properly.

I worked with daily return-based data built around the Nifty 50 index and a small set of major stocks closely linked to it. But even from the start, I was not thinking of this as many separate experiments. I wanted the project to move like one connected line of thought, where one result creates the next doubt, and that next doubt pushes the work further.

That is why I began with regression. First, I needed to know whether there was any real explanatory relationship there at all. Then the next issue came up. Even if that relationship exists, is predicting the exact return value really the right way to look at the problem, or is market direction more stable? From there, the project moved into tree-based and ensemble models to check whether added complexity was actually revealing deeper structure, or only making the model look smarter. In the last part, I moved beyond simple model scores and looked at reliability, because a model should not be called useful just because one number looks strong.

So in the end, this project became less about comparing models and more about understanding what kind of ML structure actually fits this dataset, where that usefulness starts becoming real, and where it starts becoming weaker.

2. Problem Statement

Daily financial data is available in large quantity, but that does not automatically mean the relationship inside it is useful. In market data, especially return-based data, patterns can start appearing very quickly. But the real issue is not whether a pattern can be seen once. The real issue is whether that pattern still holds when the model is pushed onto unseen data. That was the actual starting problem behind this project.

I wanted to understand whether the daily movement of the Nifty 50 index could be explained through the return behaviour of a small set of major constituent stocks, and whether ML could capture that relationship in a way that was not just visible in output, but also meaningful when looked at properly. For me, this was not only a prediction

problem. It was also a structure problem. Is there a real signal here? If yes, then how much of it can actually be captured? And what kind of model is really making sense of it?

Another issue was complexity itself. In projects like this, it is very easy to assume that a more complex model will automatically give a better answer. But financial data does not work that simply. Noise can also get picked up very easily and start looking like intelligence. So I also wanted to test whether increasing complexity was actually improving the learning, or whether a simpler model was already doing the more dependable job.

So the problem in this project was never just whether ML can predict market-related behaviour. The deeper problem was whether it could find a useful structure in the data, hold that structure on unseen observations, and do that in a way that stays understandable instead of only looking impressive.

3. Dataset and Project Setup

For this project, I worked with daily return-based financial data built around the Nifty 50 index and five major stocks: HDFC Bank, ICICI Bank, TCS, Infosys, and Reliance. I did not want to take too many stocks from the beginning, because that would make the setup wider but less clear. I wanted a smaller, more meaningful set, so the relationship with the index could be examined properly before adding anything extra.

The target in this project was the daily return of the Nifty 50 index. The feature side consisted of the daily returns of those five stocks. So each row in the dataset represented one trading day, with stock returns as the input and the Nifty return as the output. That gave the project a simple supervised learning structure from the start.

I did not use raw price levels here. I used returns because prices of different securities sit on very different scales, which can make comparisons less useful. Returns put the movement into a more comparable form. Since the project was about understanding relationships and signals, it made more sense to work with them than with price values.

I also kept a separate training and testing setup. For me, that was not just a formal ML step. It mattered because a model can look good on the data it has already seen and still lose its strength when it is tested on unseen observations. Since the main point of this project was to determine whether any useful structure is actually present, I had to verify whether that structure still holds outside the training sample.

The setup was kept controlled on purpose. I did not want to begin by adding too many engineered variables or making the project more complex than needed. First, I wanted to see whether a small set of understandable features could already show something

meaningful. If the signal is not visible there, then adding more complexity too early can make the project look smarter without actually making it clearer.

The project setup is built using view-based structures rather than static tables. This allows the pipeline to remain dynamic, so that when new data is added, it automatically flows into the evaluation layer without requiring manual restructuring. In that sense, the project is not fixed to a single snapshot, but can continue updating as new observations become available.

- **Target variable:** Nifty 50 daily return
- **Feature variables:** Daily returns of HDFC Bank, ICICI Bank, TCS, Infosys, and Reliance
- **Data structure:** One row per trading day
- **Learning setup:** Supervised learning format
- **Evaluation check:** Separate train and test segments
- **Design choice:** Keep the starting setup small, controlled, and interpretable

4. Analytical Path / Project Method

I never wanted this project to become a case where different models are run one after another and then judged only by putting their scores side by side. For me, the project had to move like one connected thought process. One stage had to answer something real, and that answer had to create the next question. So the method here was never just about applying models. It was about seeing how the problem changes once one part of it starts becoming clearer.

That is why I began with linear regression. Before doing anything else, I first needed to know whether the daily return of the Nifty 50 was showing any meaningful explanatory relationship with the selected stock returns. If that first relationship itself was not visible, then there was no point in moving too quickly toward more complex ML.

Once that part was clear, the next issue became more important. Even if some explanatory power exists, predicting the exact return value can still remain quite sensitive to day-to-day volatility. That is where the project shifted a little. Instead of staying with exact return magnitude, I moved toward classification and asked whether market direction itself could be captured in a more stable way.

After that, one more doubt came up naturally. If the directional setup is working, then is the signal mostly linear, or is there some nonlinear and conditional structure inside it as

well? That is what took the project toward a tree-based structure. However, a standalone decision tree model was not directly available in BigQuery ML in the form required for this stage. Because of that, this part of the analysis was carried out using a **boosted tree model with controlled depth (max depth = 3)**.

This was done deliberately. I did not want to move into full model complexity at this point. By keeping the depth limited, the model behaved closer to a simple, interpretable tree-like structure, while still working within the available framework. This allowed me to test whether any basic nonlinear or conditional patterns exist in the dataset, without allowing the model to become too flexible too quickly.

From there, the next question followed naturally. If a controlled boosted-tree setup is not adding much beyond classification, then what happens when the model is allowed to become more flexible? That is what led to the next stage, where the model moved into a broader ensemble structure. But even there, the point was not to assume that more complexity must mean better learning. The real test was whether that extra flexibility was actually uncovering something stronger, or only fitting the data more tightly.

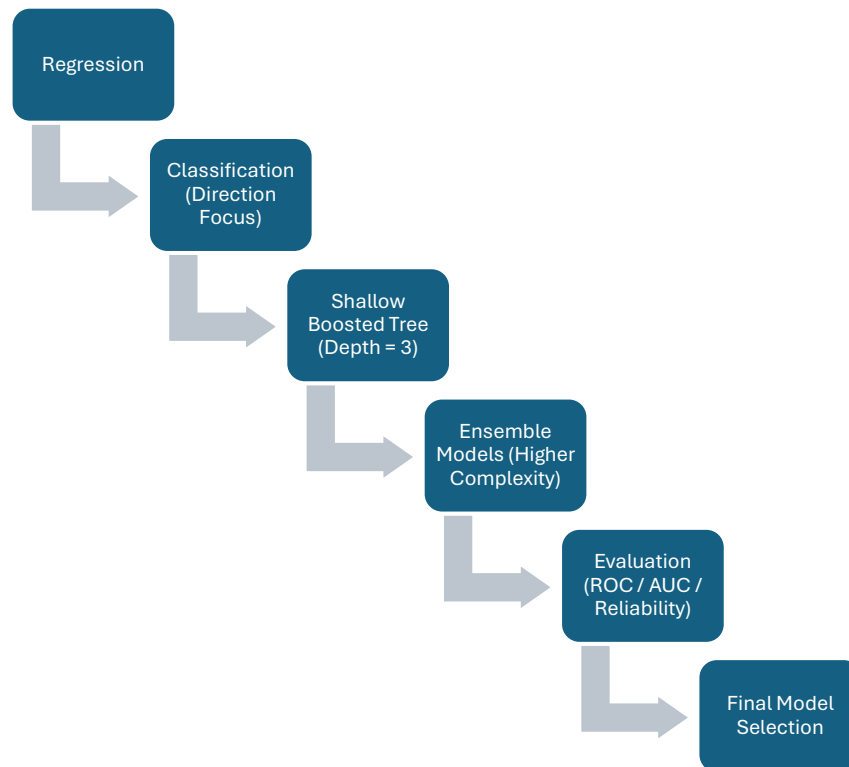
Then one final issue remained. If a few models give reasonably close results, then how should the final choice actually be made? Accuracy alone is not enough in that case. That is why the last part of the project moved into ROC-AUC, class-level evaluation, and reliability of predicted probabilities. I did not want the final model choice to come from one number alone. I wanted it to come from the overall way the model behaves when it faces unseen data.

So the project moved in this order:

- **Stage 1:** Regression to check the first explanatory relationship
- **Stage 2:** Classification to test whether direction is more stable than exact return value
- **Stage 3:** Shallow boosted-tree modelling (max depth = 3) to check for nonlinear and conditional structure
- **Stage 4:** Broader ensemble modelling to test whether added complexity creates real value
- **Stage 5:** Reliability-focused evaluation to support the final model choice

That is how the project moved from beginning to end. Each stage came from a real doubt, and the project only moved ahead because the earlier answer was not enough on its own.

Figure 1. Step-wise Analytical Flow of the Project from Regression to Final Model Selection



5. Major Question 1 — Can regression act as the first lens for this dataset?

5.1 The Question

The first major question of the project had to begin from a basic place. Before moving toward classification, trees, or anything more complex, I first needed to understand whether the selected stock returns were able to explain the daily return of the Nifty 50 in a meaningful way at all. If that first relationship itself was weak, then there was no point in pushing the project forward too quickly.

So the first question was simple: can regression be used as the first lens for this dataset?

5.2 Why This Stage Was Needed

Regression, for me, was like the first checkpoint. It is the simplest way to see whether changes in the target can be understood through changes in the features, and whether a basic relationship between the index and the selected stocks is visible in a direct way. If regression works, it gives useful information. And even if it does not work, that is still

useful, because it would mean the relationship is more noisy or more complex than it first appears.

So this stage mattered because I did not want to assume complexity from the start. I first wanted to see whether the data was already showing something meaningful in a simpler linear form.

5.3 Method Used

For this stage, I used linear regression with the daily return of the Nifty 50 index as the target variable and the daily returns of HDFC Bank, ICICI Bank, TCS, Infosys, and Reliance as the feature variables. The model was built to predict the same-day return of the Nifty using the same-day returns of these five stocks.

That setup also made sense economically. Nifty 50 is made up of 50 stocks, and these five stocks hold a meaningful weight in the index. So if their same-day movement is already able to explain a large part of the same-day index movement, that itself would be a useful first signal.

The model was checked on both training and test data. For me, that mattered because the result should not look good only on the fitted sample. It should also remain meaningful when moved onto unseen data.

5.4 Key Findings

The regression results were strong. The training R-squared was about **86%**, and the test R-squared stayed around **82%**. That gap is there, but it is still controlled, which means the relationship does not collapse when it moves into unseen data.

Residuals also became important here. The point was not only to see whether the model fits well, but also to see where it fails and what that failure says. The training residuals were somewhat higher on average than the test residuals, and one important reason for that is the nature of the training period itself. The training data included the COVID shock period, which carried much stronger volatility than the later test period. So the model had to absorb more extreme situations while trying to reduce the error on average.

That means the higher training residual does not automatically weaken the model. It also reflects the fact that the training sample had to cover a more difficult volatility environment.

Overall, the results still showed a clear and direct relationship between the selected stocks and the Nifty index.

5.5 Interpretation

For me, this stage was important not because regression was supposed to become the final answer, but because it had to tell me whether the project had a valid starting base or not. And it did. The selected stock returns were able to explain a meaningful part of the Nifty movement, and the model stayed reasonably stable across train and test data. That already tells me that the dataset contains real structure.

At the same time, this stage also showed its own limit. Regression focuses on the exact return value, and because of that, it remains exposed to volatility in a more direct way. So even though the relationship is meaningful and the model is useful, exact-return prediction still carries volatility-linked noise.

That is what made the next stage necessary. The issue was no longer whether a relationship exists. The issue became whether changing the target from exact return to direction could reduce that sensitivity and make the model more stable.

5.6 Transition

So regression did its job as the first lens. It showed that the dataset is not just noise, and that a simple model can already capture a meaningful part of the relationship. But it also showed that exact-return prediction is still affected by volatility. That is what pushed the project toward the next question: if the problem is reframed from exact return to market direction, does the result become more stable and more useful?

6. Major Question 2 — If the target changes from exact return to market direction, does the model become more stable and more useful?

6.1 The Question

Once the regression stage was done, one thing became clear. The relationship in the dataset was real, but exact-return prediction was still carrying some sensitivity to volatility. So the next question came from there quite naturally. If the target is changed from exact return value to only market direction, does the model become more stable and more useful?

That was the second major question of the project. The point was not just to move from one model to another. The real point was to change the form of the problem itself and see whether that makes the signal easier to capture in a more dependable way.

6.2 Why This Stage Was Needed

Regression had already shown that the dataset contains meaningful structure. But it had also shown its limit. In financial data, especially daily return data, exact value prediction can remain sensitive because even when the relationship is real, day-to-day volatility still affects the magnitude. So the next logical step was to ask a slightly different question.

Instead of asking how much the market moved, I wanted to ask whether the market moved up or down. That changes the problem in an important way. The model is no longer being asked to hit the exact number. It is being asked to identify the correct direction.

That mattered because if the signal in the dataset is more stable in directional form than in exact-value form, then classification would not just be another model step. It would actually be a better framing of the problem.

6.3 Method Used

For this stage, I used logistic regression as the first classification model. The feature side remained the same as before: daily returns of HDFC Bank, ICICI Bank, TCS, Infosys, and Reliance. What changed was the target.

Instead of predicting the exact daily return of the Nifty 50, the target was converted into a binary directional variable. Positive-return days were treated as one class, and negative-return days were treated as the other. So the model was no longer trying to answer how much the market moved. It was trying to answer which direction the market moved.

That shift matters a lot here, because regression and classification are not solving the same problem. Regression predicts a value. Classification predicts the probability of belonging to a category. So once the target was changed, the project was also asking a different kind of question.

6.4 Key Findings

The classification result was strong, and more importantly, it stayed stable. Accuracy remained strong on both train and test data, and the model did not show the kind of weakening that would have raised concern. In fact, the result suggested that once the target was reframed into direction, the structure in the dataset became easier to hold in a dependable way.

This mattered because it showed that the model was not just reacting to random movement. It was capturing a directional pattern that remained meaningful even outside the training sample.

Compared with regression, the classification setup looked more stable. That does not mean regression failed. It means the directional version of the problem appears to fit the dataset better than the exact-return version.

6.5 Interpretation

For me, this was one of the most important turning points in the whole project.

Regression had already shown that the data contains signal. But classification showed something more specific: the signal becomes more usable when the problem is framed through direction instead of exact return magnitude. That is a meaningful difference.

This also helped clarify something deeper. Regression and classification should not be compared as if they are doing the same job. They are not. Regression is trying to estimate the exact value. Classification is trying to judge the right category. In this dataset, the category-based version of the signal looked more stable, more dependable, and more practical.

So this stage did not only show that classification performs well. It also showed that the dataset itself may be better understood through directional logic than through exact-return logic. That was the real value of this stage.

6.6 Transition

Once logistic classification showed that directional prediction is more stable and more useful than exact-return prediction, the next question became sharper. If this directional signal is working so well, then what kind of signal is it actually? Is it mostly linear, or is there some conditional and nonlinear structure inside it as well?

That is what pushed the project toward the next stage: moving into a boosted tree-based modelling approach with controlled depth, to check whether any additional structure exists beyond the linear classification framework.

7. Major Question 3 — Does a shallow boosted-tree model (depth = 3) reveal any conditional nonlinear signal beyond classification?

7.1 The Question

After the classification stage, one thing became clear. The directional signal was working well and remained stable. But that also raised another doubt. If the model is already capturing direction properly, then is this signal mostly following one broad rule, or is there some conditional structure inside the data that is not being captured yet?

So the next question became simple: does moving toward a shallow boosted-tree structure reveal something more, or is the signal already mostly visible in the linear form?

7.2 Why This Stage Was Needed

This stage was needed because a good classification result does not automatically mean that the relationship is fully understood. Logistic regression works through a single global relationship. But financial data does not always behave in that way. There can be situations where the relationship changes depending on conditions.

So I wanted to check whether the dataset contains that kind of conditional behaviour. If it does, then a tree-based structure should be able to capture it better by splitting the data into smaller segments. But if it does not, then even after moving to a nonlinear structure, the result may not change much.

7.3 Method Used

The original intention was to test a simple decision tree model. However, in BigQuery ML, a standalone decision tree model was not available in the required form for this stage. Because of that, the model used here was a **boosted tree model**.

To keep this stage controlled, I restricted the model to a **maximum depth of 3**, so that it behaves like a shallow structure rather than a highly complex ensemble. The idea was to keep the model simple enough to act as a first nonlinear check, without allowing it to become too flexible too quickly.

So even though the model is technically an ensemble, its behaviour at this stage is closer to a controlled, shallow boosted-tree structure. Although technically an ensemble model, it is used here as a controlled shallow tree proxy due to platform limitations.

7.4 Key Findings

The model remained stable across training and testing. Accuracy stayed strong, and the gap between train and test was controlled. So the model did not break once it was applied to unseen data.

At the same time, some differences in feature behaviour started to appear. Certain features were more effective in splitting the data than others, which suggests that the relationship is not completely uniform. So there are some conditional effects present in the dataset.

But when I compared this with the earlier classification result, the improvement was not clear. The model was stable, but it did not add anything meaningfully stronger than what logistic regression had already shown.

So while the shallow boosted-tree structure did reveal some variation in how features behave, it did not change the overall conclusion about the signal in a strong way.

Table 1. Confusion Matrix for Logistic Classification on Test Data

Actual / Predicted	Predicted Up (1)	Predicted Down (0)
Actual Up (1)	240 (TP)	36 (FN)
Actual Down (0)	34 (FP)	201 (TN)

The confusion matrix shows that most predictions are correctly classified, with errors distributed in a relatively balanced way between false positives and false negatives. This indicates that the model is not biased toward one direction and supports the overall stability observed in the classification results.

7.5 Interpretation

For me, this stage gave a mixed answer.

It showed that the dataset is not purely linear. There are some conditional patterns, and different features behave differently depending on how the data is split. So the relationship is not completely uniform.

But at the same time, that effect was not strong enough to change the practical outcome. The simpler classification model was already capturing most of the useful signal. The shallow boosted-tree model did not add enough extra value to justify moving away from it.

So the answer here is not that nonlinear structure does not exist. It is that, in this dataset, it is not strong enough to change the decision at this stage.

7.6 Transition

So this stage did not close the question completely. It only refined it. If a shallow boosted-tree structure is not adding much, then one possibility is that the depth is too limited. The next step, then, is to see what happens when the model is allowed to become more flexible.

That is what leads to the next stage: moving into a more open ensemble structure.

8. Major Question 4 — If model complexity is increased further, does it reveal stronger signal or only create stronger-looking confidence?

8.1 The Question

After the previous stage, the situation was still not fully clear. The shallow boosted-tree model had shown that some conditional structure exists, but it had not changed the overall result in a strong way. That raised another doubt. If the model is allowed to become more flexible, will it finally capture something stronger, or will it only start fitting the data more closely without adding real value?

So the question here was simple: does increasing model complexity actually improve the usefulness of the model, or does it only make the model appear stronger?

8.2 Why This Stage Was Needed

This stage was needed because the earlier result could be interpreted in two ways. One possibility is that the dataset does not contain strong nonlinear structure. The other possibility is that the earlier model was too controlled to capture it fully.

Since the shallow boosted-tree model was restricted in depth, it was still acting as a limited structure. So it was not enough to conclude that complexity does not help. It was still possible that a more flexible model could extract something additional from the data.

So this stage was about testing that directly. Instead of assuming that complexity will help, I wanted to see whether it actually changes the outcome in a meaningful way.

8.3 Method Used

At this stage, the model was allowed to move beyond the shallow boosted-tree setup into a more flexible ensemble structure. The restriction on depth was relaxed, and the model was allowed to capture more detailed patterns in the data.

The idea here was not to maximise performance blindly, but to observe what changes when the model becomes more flexible. If the dataset contains deeper structure, then this stage should reveal it. But if it does not, then the model may simply start fitting the data more tightly without improving its real usefulness.

8.4 Key Findings

The models remained strong and stable overall. They did not collapse on the test data, and accuracy stayed close to the earlier models.

But one clear change appeared. The model started producing more extreme probability values. In simple terms, it became more confident in its predictions.

At first, that can look like an improvement. But when I looked at the results more carefully, that confidence was not always reliable. Along with more high-confidence correct predictions, there were also more high-confidence wrong predictions.

So the model was becoming more forceful, but not necessarily more dependable.

8.5 Interpretation

For me, this stage made one thing clearer. Increasing complexity does not automatically improve the usefulness of the model.

The model was able to push its predictions more strongly, which means it was capturing something additional in the data. But that did not translate into a better practical outcome. The overall performance remained similar, and the risk associated with confident mistakes increased.

So the result here is not that ensemble models are ineffective. It is that, in this dataset, the additional complexity is not creating enough value to justify the added risk.

The simpler model was already capturing most of the useful signal, and increasing complexity mainly changed how strongly the model expresses its predictions, not how correctly it behaves.

8.6 Transition

So this stage brought the project to a different kind of problem. Now the question was no longer about whether a model works or not. Multiple models were already working well.

The real question now became: when several models look strong, how should the final choice be made?

That is what leads into the final stage, where the focus shifts from performance to evaluation and reliability.

9. Major Question 5 — If several models look strong, which one is actually the most reliable?

9.1 The Question

By the time I reached this stage, the project had become more difficult in a different way. Earlier, the problem was whether the dataset contains meaningful structure at all. But now the problem had changed. Multiple models were already showing strong results, and the difference between them was no longer large enough to be settled through accuracy alone.

That is what created the fifth and final major question of the project: if several models look strong on the test data, then which one should actually be trusted more? And can probability-based evaluation reveal something that fixed-threshold accuracy cannot show?

9.2 Why This Stage Was Needed

This stage became necessary because accuracy had reached its limit. Accuracy works at one threshold, usually 0.50. Because of that, two predictions with very different confidence levels can still be treated in the same way. A model giving 0.51 and another giving 0.91 may both be marked correct, even though one is uncertain and the other is highly confident. So accuracy alone could no longer explain the real difference between the models.

From there, I moved into ROC analysis. That helped because ROC shows how the model behaves across many thresholds instead of just one. But even then, it still remained difficult for me to use ROC directly for final judgment, because it produced too many values across too many thresholds. I could see that logistic was showing strong separation, but I still did not feel fully satisfied with that alone. That is what led me to AUC.

AUC helped because it summarized the full ROC behaviour into one value. That made comparison easier. But then another important thing happened. Once I looked at system-level AUC, the gap between the models became narrower than what I had earlier understood from threshold-based judgment. So even AUC improved the understanding, but it still did not fully settle the final choice. That is where the real issue changed again — from confidence to reliability.

9.3 Method Used

At this stage, I did not build a new model. Instead, I built a deeper evaluation layer for the models already created in the project: Logistic Regression, shallow boosted-tree model

(max depth = 3), the Controlled Ensemble, and the Unrestricted Ensemble. The target and feature structure stayed the same. What changed was the evaluation framework.

The evaluation moved through four layers:

- **ROC analysis**, to study threshold-wise separation through TPR and FPR
- **AUC analysis**, to summarize ranking ability across thresholds
- **Probability distribution analysis**, to see how aggressively or cautiously each model expresses confidence
- **Reliability analysis**, to study how each model behaves when it is confident, especially when it is wrong

For reliability, I selected the common threshold range where the models were showing strong separation in the test data. The main threshold points fell around **0.50 to 0.60**, so I used **0.50, 0.55, and 0.60** for the reliability analysis. Then I created confidence bands. Predictions above **70%** or below **30%** were treated as high-confidence predictions, while values between **30% and 70%** were treated as low-confidence predictions.

9.4 Key Findings

The first important result was that ROC and AUC did improve the comparison, but they did not create a dramatic final separation. Logistic showed the strongest ROC separation overall, while the shallow boosted-tree and ensemble models remained close. Test AUC values for all four models were also very close, with Logistic around **0.93** and the others around **0.92 to 0.93**. So all models were clearly strong in ranking terms.

The next result came from probability behaviour. The ensemble models showed more extreme probabilities. That made them look more forceful and more confident. Logistic regression, in comparison, remained more balanced and controlled. At first glance, stronger confidence can look better. But that is not the full story.

The clearest answer came from reliability analysis. On unseen test data, the more complex models showed more high-confidence correct predictions, but they also showed more high-confidence wrong predictions. Logistic behaved differently. Its high-confidence correct predictions did decrease on test data, but instead of turning into many high-confidence wrong predictions, a larger part of that shift moved toward low-confidence correct predictions. In simple terms, it became more cautious rather than more dangerously confident. That became the key difference.

The final numbers support that clearly. Logistic produced the **lowest high-confidence wrong rate**, at about **5.4%** on the test data. The shallow boosted-tree model stayed

slightly above that at about **5.6%**, while both ensemble models were higher at around **7.2%**. So even when the ensemble models looked stronger in confidence, they were also carrying higher danger when they were wrong.

9.5 Interpretation

For me, this stage gave the clearest and most complete answer in the whole project.

It showed that final model choice cannot stop at accuracy alone. It also cannot stop fully at AUC, because even AUC can leave several models looking close. The real difference appears when confidence is judged together with safety. A model that looks stronger because it gives more extreme probabilities is not automatically better. What matters more is whether that confidence remains dependable on unseen data.

That is why minimizing high-confidence wrong predictions mattered more to me than maximizing high-confidence correct predictions. A model that stays a little more cautious on unseen data is safer than a model that remains aggressive and makes more confident mistakes. From that point of view, logistic regression gave the most balanced answer. It stayed strong, but it also adjusted conservatively on unseen data. That made it more trustworthy for practical use.

So the final answer was not that the more complex models were weak. They were still strong. But they were not the most reliable. Logistic regression remained the most dependable model for this dataset because it combined strong overall performance with the safest confidence behaviour.

9.6 Transition

So the fifth major question closed the final uncertainty of the project. After this stage, the answer was no longer only about whether ML can find signal in daily financial data. The answer became more precise: yes, the signal is real, but the model that handles it best is not the most complex one. It is the one that stays strong while remaining more controlled and more reliable.

That final result now leads into the overall project conclusion.

10. Final Model Selection

By the time I reached this point, the situation was not about finding a model that works. Multiple models were already working well. Regression had shown that the dataset contains a real relationship. Classification had made that relationship more stable by focusing on direction. The later stages had tested whether more complexity could add anything beyond that.

So the decision was no longer about performance in isolation. It became a question of which model remains more dependable when all aspects are considered together.

Logistic regression stayed consistent across train and test data. It was able to capture the directional signal clearly without becoming too sensitive to changes in the data. When I moved toward shallow boosted-tree and ensemble models, the behaviour changed slightly. The models became more confident in their predictions, but that confidence was not always reliable.

What stood out more clearly in the final evaluation was how the models behaved when they were wrong. The more complex models produced more high-confidence wrong predictions. Logistic behaved differently. It remained more controlled, and when it adjusted on unseen data, it did so by becoming more cautious rather than more aggressive.

That difference became more important than small differences in accuracy or AUC. The goal here was not to choose the most complex model, but to choose the one that remains more dependable.

So the final model selection was based on overall behaviour, not just performance metrics. In this dataset, logistic regression provided the best balance. It captured the signal well, remained stable on unseen data, and avoided the kind of risk that appeared in more complex models.

That is why logistic regression was selected as the final model.

11. Final Metrics Snapshot

This section presents the final comparison of model performance based on the most relevant evaluation metrics. The purpose is not to show every possible number, but to provide a clean snapshot of how the models performed, especially on unseen data.

The models compared include Logistic Regression, the shallow boosted-tree model, and the ensemble-based models. While all models showed strong overall performance, the differences become clearer when stability and reliability-related metrics are considered together.

Table 2. Final Model Metrics Snapshot

Model	Test Accuracy	Test AUC	High-Confidence Wrong Rate
Logistic Regression	~0.86	~0.93	~5.4%
Shallow Boosted-Tree Model (Depth = 3)	~0.84–0.85	~0.92–0.93	~5.6%
Ensemble Models	~0.84–0.85	~0.92–0.93	~7.2%

The table shows that while accuracy and AUC remain close across models, the difference becomes more visible in the reliability-related measure. Logistic regression maintains the lowest high-confidence wrong rate, which supports its selection as the final model.

12. Overall Interpretation

Looking at the project as a whole, one thing becomes clear. Machine learning can be applied meaningfully to daily financial data, but the usefulness does not come from complexity alone. It comes from how the problem is framed and how the model behaves when tested properly.

The first important learning is that the dataset itself contains real structure. The relationship between a small group of major stocks and the Nifty 50 index is not random. It can be learned in a stable way, even on unseen data. That establishes that ML is not being forced onto noise in this case.

The second learning is that how the problem is defined matters a lot. When the task was framed as exact return prediction, the model remained sensitive to volatility. But when the same problem was reframed into direction, the signal became more stable and more usable. This shows that in financial data, the form of the target can matter as much as the model itself.

The third learning is about model complexity. Moving from linear classification to shallow boosted-tree and then to ensemble models did introduce more flexibility, but it did not create a clear improvement in practical usefulness. The models became more confident, but that confidence did not always translate into safer predictions. So complexity alone did not improve the quality of the final outcome.

The final and most important learning is about reliability. A model should not be judged only by how often it is correct, but also by how it behaves when it is wrong. In this project, the simpler model remained more balanced on unseen data, while more complex models became more aggressive. That difference matters more than small differences in accuracy.

So overall, the project shows that machine learning can extract meaningful signal from daily financial data, but the best results come from choosing the right problem structure and the most dependable model, not the most complex one.

13. Limitations

While the project shows that meaningful relationships can be extracted from daily financial data, it also has clear limitations that need to be acknowledged.

The first limitation comes from the feature structure. The model uses same-day returns of selected stocks to explain the same-day movement of the Nifty 50. While this helps in understanding relationships, it limits the predictive depth of the model, because it does not capture lag effects or forward-looking signals.

The second limitation is the absence of broader market and macro variables. The model relies only on a small set of stock returns, without incorporating factors such as interest rates, global indices, sector-wide movements, or macroeconomic indicators. This keeps the setup clean, but also restricts the model's ability to capture wider influences on market behavior.

Another limitation is that the dataset is not explicitly designed to capture different market regimes. Financial markets behave differently during stable periods and high-volatility periods, but the current setup does not separate or model these regimes differently.

Finally, the project focuses on a specific target design based on daily direction. While this improves stability, it also simplifies the problem, and may not fully capture more complex aspects of market movement.

These limitations do not reduce the value of the current findings, but they define the boundaries within which the results should be interpreted.

14. Future Improvements

The current project establishes a clear base, but there are several ways in which it can be extended to improve both depth and practical usefulness.

One natural extension is the inclusion of lag-based features, where past return values are used along with current data to capture delayed market effects. Instead of relying only on same-day movements, this would allow the model to learn how earlier movements influence current outcomes and improve its predictive structure.

Another improvement is the addition of broader market and macro variables. Factors such as interest rates, global indices, or sector-level indicators can provide a wider context and help the model capture influences that are not present in a small set of stock returns.

The project can also be extended by incorporating volatility-related features, such as rolling standard deviation or volatility regimes. This would allow the model to better adapt to changing market conditions instead of treating all periods in the same way.

In addition, a more structured approach to regime detection can be introduced. Separating stable and high-volatility periods and modeling them differently may improve the reliability of the results.

Finally, further work can focus on improving feature design rather than only increasing model complexity. The current project already suggests that better feature structure may add more value than simply using more advanced models.

These improvements would help extend the project from a controlled analytical setup toward a more practical and robust financial modelling framework.

15. Final Answer to the Problem Statement

The question at the start of this project was whether daily financial data can actually be understood in a meaningful way using machine learning, or whether it only appears useful when looked at through model outputs.

From the way the project has progressed, the answer is yes, but not in a simple way.

The dataset does contain a real relationship. A small set of major stocks is able to explain a large part of the Nifty 50 movement, and that relationship remains stable when tested on unseen data. So this is not a case where machine learning is being applied to noise.

But the way the problem is framed matters a lot. When the focus was on exact return prediction, the results stayed sensitive to volatility. When the same setup was shifted toward direction, the signal became more stable and more usable. That change itself explains a large part of the outcome.

After that, increasing complexity did not change the result in a strong way. The models became more confident, but not more reliable. The simpler model was already capturing most of what the dataset was offering.

So the answer here is not just that machine learning works. It is that it works better when the problem is structured carefully, and when the model is chosen based on how it behaves, not just how it performs.

In this case, the most dependable result came from the simpler model, not the more complex one.

16. Appendix / Note on Supporting Assets

The main report presented here is based on a detailed working document developed during the actual execution of the project. This underlying document captures the step-by-step analysis, observations, and interpretation as the models were built and evaluated in real time.

That document is included in the project repository as a supporting file. It reflects the original thought process and progression of the project, from initial exploration to final conclusions.

In addition, the repository also contains supporting assets such as datasets, model outputs, and additional tables or charts where relevant.